Age Rating Prediction and Explicit Content Detection for Movie Scripts

Jake Tovey
22018158

UXCFXK-30-3


UWE
Bristol
University
of the
West of
England

Abstract

This project will be using machine learning to read through full movie scripts and provide a predicted age rating along with flagged sections of the script containing the most influential lines that contributed to this age rating. It will be learning the context of the language used and age rating classification criteria to do this. My program will take inputs of excerpts from movie scripts, and scraping tools will be used to get movie scripts with age ratings attached.

Table of Contents

Table of Figures

Introduction

The field of Natural Language Processing (NLP) is one that has been studied for decades and recently, with the explosion of machine learning technology, has gotten even more advanced and utilised in modern life. In the modern day, AI tools such as ChatGPT use deep neural networks to discern the meaning behind human speech to provide helpful responses. Countless companies look to chatbots to provide a similar service, and AI is used for content detection on platforms like YouTube. This leads to this project – a machine learning model to accurately predict a movie's age rating based on its script and flag explicit content. Technology for flagging explicit language has been around for decades, with a lot of chatbots making use of rule-based heuristics to censor language in the early 2000s, leading to the convenience of tools such as Python's Profanity library today. The

advancement of deep learning in the 2010s began to capture the relationship between words, advancing sequential text analysis to capture the contextual meaning of words. Some scenes in movie scripts may be sexual or violent in nature without directly stating so, and it is the aim of this project to use these NLP techniques and systems to capture the contextual understanding of a string of text. Techniques like topic modelling have advanced such that themes and topics can be identified within text, which is very useful for this project.

Several past attempts at a tool like this have been encountered during research, some regarding the detection of obscenities in music, and others identifying sexual content in provided text, but few directly aligned with this project's proposal. Only one was found, in fact, regarding movies and age rating prediction, though its end goal differed to that of this project. While plenty of attempts to capture the contextual meaning behind words shave been made, this idea is novel among them and has yet to be done before. A tool like this one will be useful in the world of pre-production for movies. There are many instances of filmmakers making movies and then having to cut scenes or lines to adjust to a desired age rating after manual review. The end goal of this project is to eliminate this waste of time and resources by providing a system that can give an accurate prediction for an age rating from just the script during the pre-production stage, highlighting the sections of the script that led to this conclusion. This will help filmmakers and writers make script adjustments before committing to filming scenes, hopefully streamlining this aspect of filmmaking and leading to saved money and time.

The primary objectives of the project are:

- To preprocess and tokenize large-scale movie script data for machine learning training and fine-tuning.
- To experiment with various transformer models (such as BERT and Longformer) and choose the most suitable for long-document classification.
- To train a model that classifies movie scripts into age ratings using a supervised learning approach.
- To evaluate the model's accuracy using appropriate performance metrics and refine based on results.
- To explore model explainability—highlighting which script excerpts contribute most to its final prediction.

In approaching this project, an Agile development methodology was used, allowing the system to be developed iteratively with feedback loops between model performance and design decisions. An Agile development process will "focus on managing time to market constraints and the ability to accommodate changes during the software development life cycle" (Cao, Mohan, Xu & Ramesh, 2009), which was highly useful for this project. Early development focused on data preprocessing—handling inconsistencies in script formatting and managing token limits. Subsequent iterations tackled model selection and tuning, followed by evaluation and refinement based on accuracy, F1 score, and class imbalance handling.

The outcome of the project is a functioning prototype that can classify a movie script into a UK age rating with encouraging levels of performance. It also highlights problematic or rating-defining content segments within the script, offering practical utility to filmmakers in pre-production. This work not only demonstrates how modern NLP can be used in creative industries but also opens the door to further work in automated content moderation and film compliance tools.

The structure of the rest of the report is as follows:

- **Chapter 2: Literature Review** – A summary of the existing research and technologies relevant to age rating prediction, NLP, and transformer-based models.
- **Chapter 3: Methodology** – A detailed explanation of the Agile methodology applied, development workflow, and decision-making process.
- **Chapter 4**: **Design** – A detailed description of the design choices of the project accompanied by diagrams demonstrating external sources used and views on how the project will look and run.
- **Chapter 4: Implementation** – A technical breakdown of the system architecture, model training pipeline, and preprocessing techniques with test results shown alongside detailed descriptions of the challenges faced and how they were overcome.
- **Chapter 5: Evaluation** – An overall evaluation of the whole project – including research and implementation.
- **Chapter 6: Conclusion and Future Work** – A discussion of the final outcomes, limitations, and potential directions for future research and development including potential improvements had more time been allocated to the project.

Literature Review

The field of machine learning has been a rapidly expanding one in recent years and has been used for an endless variety of tasks. For this project, I am interested in its capabilities to perform sentiment analysis and understand context of speech to flag age restricted content in movie scripts. First of all, I needed to choose which language to use, which ended up being an easy choice of using Python. I naturally investigated this language first, as it is the one I have the most personal experience and familiarity with, which considering that I have not worked with machine learning before immediately removed a potential hurdle of both learning a new language as well as machine learning concepts. Upon research, I found that machine learning is widely used with Python, and that Python holds many advantages over other languages in this field. A very large part of this is due to its plethora of libraries and frameworks openly available. These libraries provide countless utilities for every aspect of machine learning, such as processing or tokenization, with a large active community constantly adding new features and fixing bugs, meaning “you can rely on it for up-to-date resources and online support” (Datacamp website, n.d.). On top of this, Python also supports libraries such as TensorFlow and PyTorch which can be used for text classification and sentiment analysis, two areas with which my project is concerned.

One other struggle I had with starting this project was finding a suitable dataset, which was trickier than I initially thought. There were very few online datasets connecting movie scripts to age ratings. I ended up using the BeautifulSoup Python library to scrape scripts from the springfieldspringfield website. I chose a random number of scripts to get, as with the capabilities of the technology I have access to, I wouldn’t be able to scrape everything from the site. This does mean that it is likely that my results will be skewed slightly due to the nature of this being a student project. After downloading the

scripts, I used Python's IMDbPY library to search for each movie script I had downloaded and append the UK age rating to the top of the file via the IMDb website. Unfortunately, there were a handful of scripts where ratings were unable to be found, meaning that they had to be discarded, but for the most part, it worked successfully. To get this dataset, I followed the methodology of Mohamed and Ha (2020) which used this same script site and Python library.

With little experience in using NLPs before, I began looking into the first fundamental step – text analysis. I investigated the key ideas behind text analysis, which is an integral part of NLPs, where text is pre-processed for the machine to better identify features of the text. I first investigated topic modelling, which is used to identify topics that crop up frequently across documents, in my case being explicit content. It organizes data into clusters without needing labelled data. This is very important for my project, as I outside of profanity, the sexual or violent content I aim to identify is contextual and not clearly labelled. Topic modelling identifies latent topics in walls of text, finding certain words fitting into certain categories, for example finding words such as "tractor", "cow", or "manure" appearing frequently may denote a text about farming. I found that BERT, a learning model I had previously heard of and will discuss later in this review was used with topic modelling via BERTopic. Upon research, I found that "text preprocessing involves 4 stages" (Gupta, Patel, 2021), which are normalization, punctuation removal, stop words removal, and lemmatization. These steps are self-explanatory, denoting the removal of irrelevant text such as punctuation or words such as "the" or "and" and removing the endings of words with the same meaning. Some obvious advantages of topic modelling is its semantic understanding of topics, as well as features like being able to handle synonyms, as well as having clearer data, clustering topics instead of vectors of individual word counts. Topic modelling, however, still isn't optimal on its own, where "people have not stopped seeking to leverage external knowledge to help the learning, from document meta-data to pre-trained word embeddings" (Zhao, H.Z., Phung, D.P., Huynh, V.H., Jin, Y.J., Du, L.D. and Buntine, W.B. (2021). I wanted to compare this to Bag of Words, another text analysis model I had come across, to see which would be a better fit for my project.

Bag Of Words works by treating text as a "bag" and taking note of multiplicity (the number of times a word shows up). As opposed to topic modelling which aim for a more semantic understanding of text, Bag of Words is only concerned with the frequency of words, meaning that, while it is simple and fast, it is limited by its vocabulary size, unlike topic modelling which is easily scalable. Bag Of Words is also easily interpretable, immediately showing the counts of certain words, which is well-suited for my project as it can easily identify and quantify profanity use which plays a specific role in age rating, though topic modelling may be better suited for the overall goal of the project, being better at detecting themes such as a scene being sexual or violent. Bag Of Words, being concerned with individual words rather than overall topics, is a lot less space efficient the larger the dataset goes. It needs to tokenize each word in a sentence and add them to a vector. Upon researching Bag of Words, I found that while it doesn't require any fine tuning and is good for sentiment analysis (finding positive or negative words and pronouns used), "the drawback is that it only works well on fairly simple tasks that don't depend on understanding the context surrounding words" (Topper, 2023). Since words are considered individually, and the spaces where they show up is disregarded, this can lead to semantics being lost. While for my project this may not matter as much, as mature language is often tied to explicit scenes, there are some instances where this is not necessarily the case, and a bag of words approach can lead to less accurate overall results. I found a research paper by Kasthuriarachchy, De Zoysa, and Premaratne using a Bag of Words approach for phrase-level semantics analysis. This paper showed that the "standard bag-of-words model does not capture the effect of negation and the contextual polarity efficiently" (Kasthuriarachchy, De Zoysa, Premaratne, 2014), meaning that if something was described as "not good", a Bag of Words model may derive positive connotations from

the recognition of the word 'good'. However, the study also showed that these models can be improved with respect to recognising negation, leading to a 10% increase in accuracy, and a further 70% increase when incorporating the context of adjectives with the effect of negation. In order to improve the model, they have used POS tagging – assigning grammatical categories to words – as well as tagging negations, when found, to any previously found words of a category that they could fit with (i.e. adjectives). While this would cause room for some mishandling, as sometimes negation could be attached incorrectly, it does add more nuance and accuracy to the system, especially paired with grammatical rules. I can see from this, that some of the drawbacks associated with Bag of Words can be negated via methods to teach It word association and grammatical rules, though this is mainly useful for sentiment analysis, which is not what my project aims to achieve, being much more focused on context and topics. Further, the study left some room for error with incorrect negation handling, which is I outlined in my requirements section as wanting to avoid. I will consider using a system like bag of words for profanity checking, giving it a dataset of profanities, though I will be using topic modelling to achieve the brunt of my project due to these considerations. Especially considering that while "a variety of preprocessing steps can be used to reduce the size and variability of free text, but they can also potentially alter the meaning of some documents" (Juluru, Shih, Murthy, Elnajjar, 2021).

Next, I began to investigate how tasks like sentiment analysis and text classification are performed. I had investigated BERT previously, which I will address later, but wanted to see the building blocks of the concepts used by it for potential use in this project. I began by looking into the PyTorch library, which makes use of tensors – multi-dimensional arrays capable of storing large amounts of data. Tensors can encode words, sentences, and even full documents as numerical data to be stored and manipulated, which is very useful for deep learning algorithms. To do this, word embedding must be performed. After this, PyTorch can be used to split a training and test dataset, which can be run across several epochs. Each epoch represents a training and validation stage for the program, meaning that the model can adjust its parameters at each epoch to get more accurate predictions, though it is important to note that relying on a specific set of training data too heavily could lead to overfitting, where the program becomes too highly tuned to its specific training set as opposed to just being accurate in general. PyTorch is also able to make use of dynamic graphs, which are graphs created during the execution of a model, displaying operations and data flow. Another alternative library I mentioned was TensorFlow, which happens to be the closest competitor to PyTorch. TensorFlow works similarly, however has some key differences in how it executes machine learning. First, it uses static graphs instead of the dynamic ones created by PyTorch, which can lead to a steep learning curve. TensorFlow is a lot more manual, requiring specific precise inputs and adjustments for distributed training, making it favoured for more complex projects. It has stronger visualization capabilities and is commonly used for production and deployment. However, while this may sound like a better choice than PyTorch, there are a few important factors to consider for this decision. The first of which is that PyTorch is much more Python-friendly, making it more ideal for me as an experienced Python coder. Moreover, it is "mostly recommended for research-oriented developers as it supports fast and dynamic training." (Kurama, 2020). It is also more "focused on the design phase, with a more user-friendly interface due to it having more functions defined by default" (Novac, Chirodea, Novac, Bizon, Oproescu, Stan, Gordan, 2022), which is more appealing to me due to my inexperience with NLP libraries. With the time constraints I have for the completion of this project, and my unfamiliarity with machine learning libraries, PyTorch sounds like a much better alternative for my project, though TensorFlow is worth looking into should I come back to a project like this with more time and experience.

I next began to investigate similar studies to what I am trying to do, and what methodologies used to accomplish them. NLP (natural language processing) has been around for decades but really took off

with machine learning in the 2000s-2010s. This is when technologies for the previously discussed word embeddings began to emerge, as well as the introduction of using vectors for machine learning. Then, in the 2010s, neural networks begun to be used as well as attention mechanisms which allowed models to focus on specific parts of input sequences. In 2017, NLP began to shift into how we know it today, with the use of transformers. Models such as ELMo and GPT emerged, which were pre-trained and used transformers and contextual word embeddings. Transformers were a massive breakthrough, as they used self-attention to weigh the importance of different words in context. This led to the release of BERT in 2018, which improved on previous transformer-based NLP models by reading text bidirectionally, allowing for a higher level of context analysis. BERT is very widely used in modern NLP, and it something I came across in many papers during my research. It specializes in sentiment analysis and text classification, which very closely relates to the subject of my project, while building on the beforementioned techniques and libraries.

In one such study by Anton, Dmitry and Preslav (2021), researchers used BERT and some of its related models such as RoBERTa to detect propaganda. This was a high-quality paper that conducted a variety of tests in different areas using BERT-like models, to "shed light on some important theoretical limitations of pre-trained BERT-style models that are inherent in the general Transformer architecture" (Anton, Dmitry, Preslav, 2021). They tested segmentation and segment labelling and found that BERT-like models hold several disadvantages, but that these can be mitigated via some simple added layers. One such limitation is that BERT was incapable of paying attention to some specific punctuation symbols such as quotation marks, which naturally harms the contextual analysis of the model. They theorized that having a hybrid symbolic-distributed representation may yield better results, however this was not tested. This study showed me that if I am considering using BERT for my project, I may need to adjust and add new layers to improve the performance of the program, which is something to consider. However, despite this study being comprehensive, there were a lot of untested solutions like the hybrid representation, which ultimately makes sections of the results and conclusion parts of the paper seem more speculative and less grounded in concrete, tested methods.

After finding this study, I searched for more studies in similar fields to my project, detecting explicit content. I wanted to see if these studies would make use of BERT and if not, what other models they may make use of. I found an article with a very similar topic to mine, finding the age appropriateness of film by Mohamed and Ha (2020). In fact, I mentioned this same study earlier, as I used the referenced dataset from this study for my own project. This paper explained in detail how it got this dataset, making it very easy for me to understand where the results were coming from and how to use this within my own project. The study made use of several different methods for this task to find the best one, BERT being one of the methods. They also made use of ELMo, one of BERT's predecessors, and used different language processing algorithms with them, such as XGBoost, a gradient boosting machine with a lot of success in real world applications. They found that using the XGBoost algorithm with an input of a deep learning model such as BERT or ELMo yielded the best results, being the "clear winner…beating more modern architectures" (Mohamed, Ha, 2020). However, the results section of this paper was incredibly small and did not explain the results in much detail. They mentioned which algorithm worked best but didn't provide much explanation or intricate detail into a lot of the proposed methods. The results tables on their own weren't very clear and didn't seem to fully support the argument they presented. With that said, I did investigate using XGBoost for my project.

XGBoost has a lot of powerful key features, namely using decision trees to iteratively refine a model, as well as being able to train these trees in parallel while using L1 and L2 regularization, minimizing the risk of overfitting. It works with BERT by receiving the vectors generated by BERT as input, to create

a powerful hybrid classification model. The first paper I reviewed speculated that a hybrid model would achieve greater results, which has been proven with this second study. However, XGBoost is not without its drawbacks, being very computationally expensive as well as needing more training time. On top of this, it is also very sensitive to hyperparameters such as learning rate, max depth, etc. which can lead to difficulty with fine tuning it. The time and computational drawbacks are much more concerning for this project, as I will only have access to limited computational power and have not much time to complete it, leading to some serious consideration regarding XGBoost.

Still interested despite this, I found another paper on using XGBoost in unison with BERT by Sharma and Joshi (2024), this time to detect sarcasm. While a little further from my project as the last paper, this study is still heavily involved with NLP and contextual analysis, making it a great reference for my own project. This study used BERT for feature extraction and XGBoost to for the classification of text into sarcastic and not sarcastic. The texts in this studies dataset were tokenized and then given to BERT to be embedded. The result was then passed to XGBoost as an input. After obtaining an accuracy rating and comparing it to other algorithms such as Random Forest and Logistic Regression, the study found that the XGBoost and BERT hybrid model was nearly 10% more accurate than its best competitors. This made me set on attempting to use a similar hybrid model in my own project, with the study showing that “this integrated model is a useful tool for deciphering complex expressions in text because it can identify minute language indicators” (Sharma, Joshi, 2024). This paper was a high-quality study that clearly outlined its testing process and showed multiple sets of results in the form of both tables and graphs. However, it is important to note that these were just preliminary findings, and as the study itself makes note of, this model would need to be continually optimised and refined with need for further testing and investigation. Despite the drawbacks of XGBoost, its capabilities demonstrated across multiple high-quality studies has piqued my interest, and I will, at the very least, be conducting further research into and attempting to integrate XGBoost with my BERT model for this project.

Finally, there are some ethical considerations to be made with this project. I will need to consider user privacy, as in the context of using this project as an application, it implies that unreleased, unfinalized scripts will be given to the model. It is important that I do not save any unnecessary data from these scripts, as this could lead to copyright issues. In that same vain, I need to be careful with my dataset. I made sure to get all my data from trusted, public access websites (springfieldspringfield and IMDb) to prevent any risk of pirated content.

The next stage of this paper will outline the functional and non-functional requirements of this project, some of which were referenced prior in this section. They will build off what I have learned from this research, considering the capabilities of the technologies I’ve studied.

Requirements

The system requirements were gathered through prototyping, analysis of similar moderation systems, and consultation of BBFC guidelines. The following tables list functional and non-functional requirements using the MoSCoW prioritization method to clearly indicate essential versus optional system features.

Table 1 - Functional Requirements

ID	Requirement (User/System)	Priority	Type
1	The system must allow users to upload movie script text.	Must	User
2	The system must process the uploaded text and return an age rating prediction.	Must	System
3	The system must display a graph of confidence scores across all age categories.	Must	System
4	The system must highlight up to 30 text segments that most influenced the prediction.	Must	System
5	The system must use a pre-trained Longformer model fine-tuned for age classification.	Must	System
6	The system should support input texts up to 4096 tokens.	Should	System
7	The system must show clear sections for user input, prediction output, and explanations.	Must	User
8	The system must not retain user input after the session ends.	Must	System
9	The system could allow users to download prediction summaries.	Could	User
10	The system must process the input and return results within 1 minute.	Must	System

Figure 1 – Sequence diagram for Requirement #4

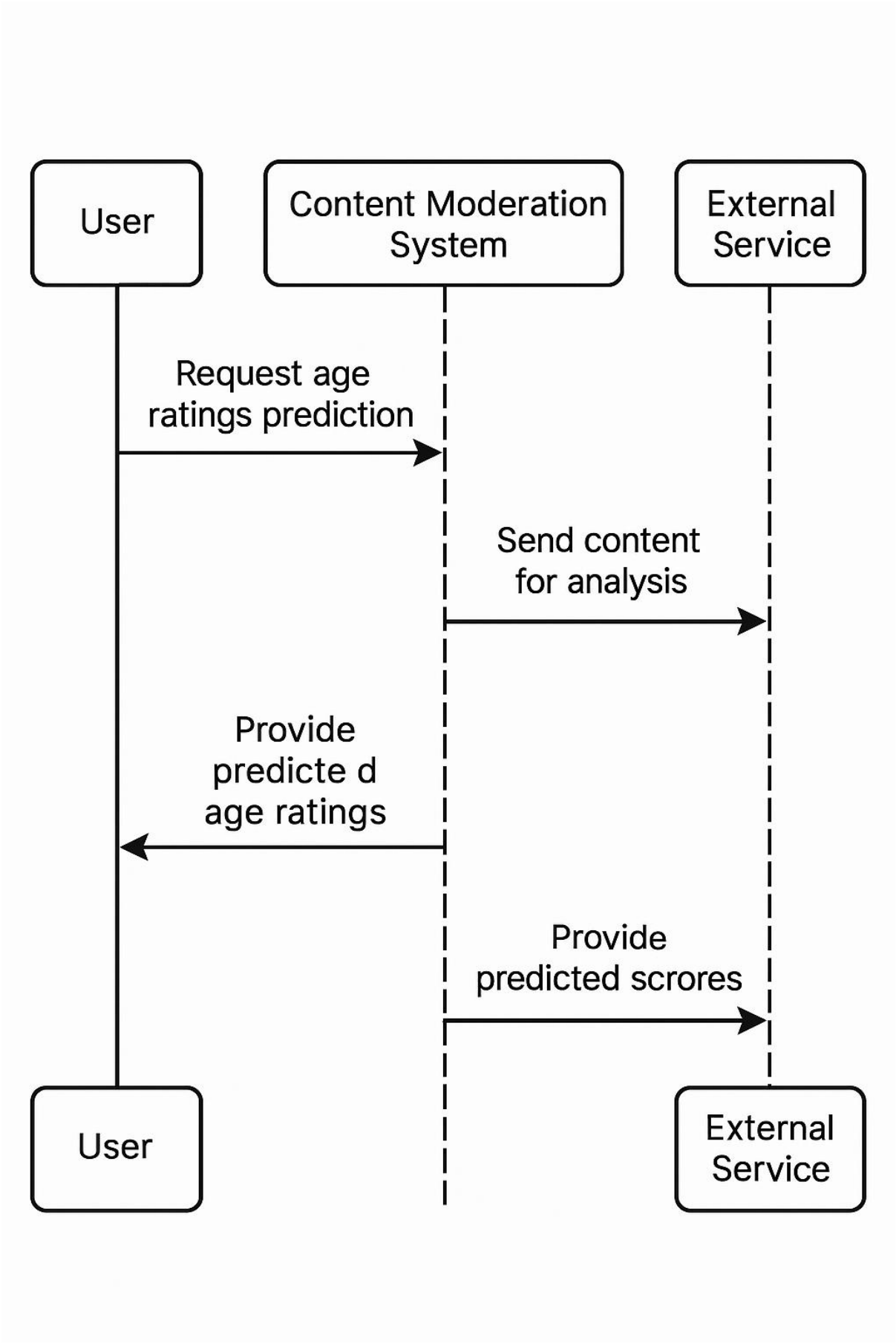
User
Content Moderation System
External Service
Request age ratings prediction
Send content for analysis
Provide predicte d age ratings
Provide predicted scrores
User
External Service

Figure 1

Figure 2 – Class diagram for Requirement #5

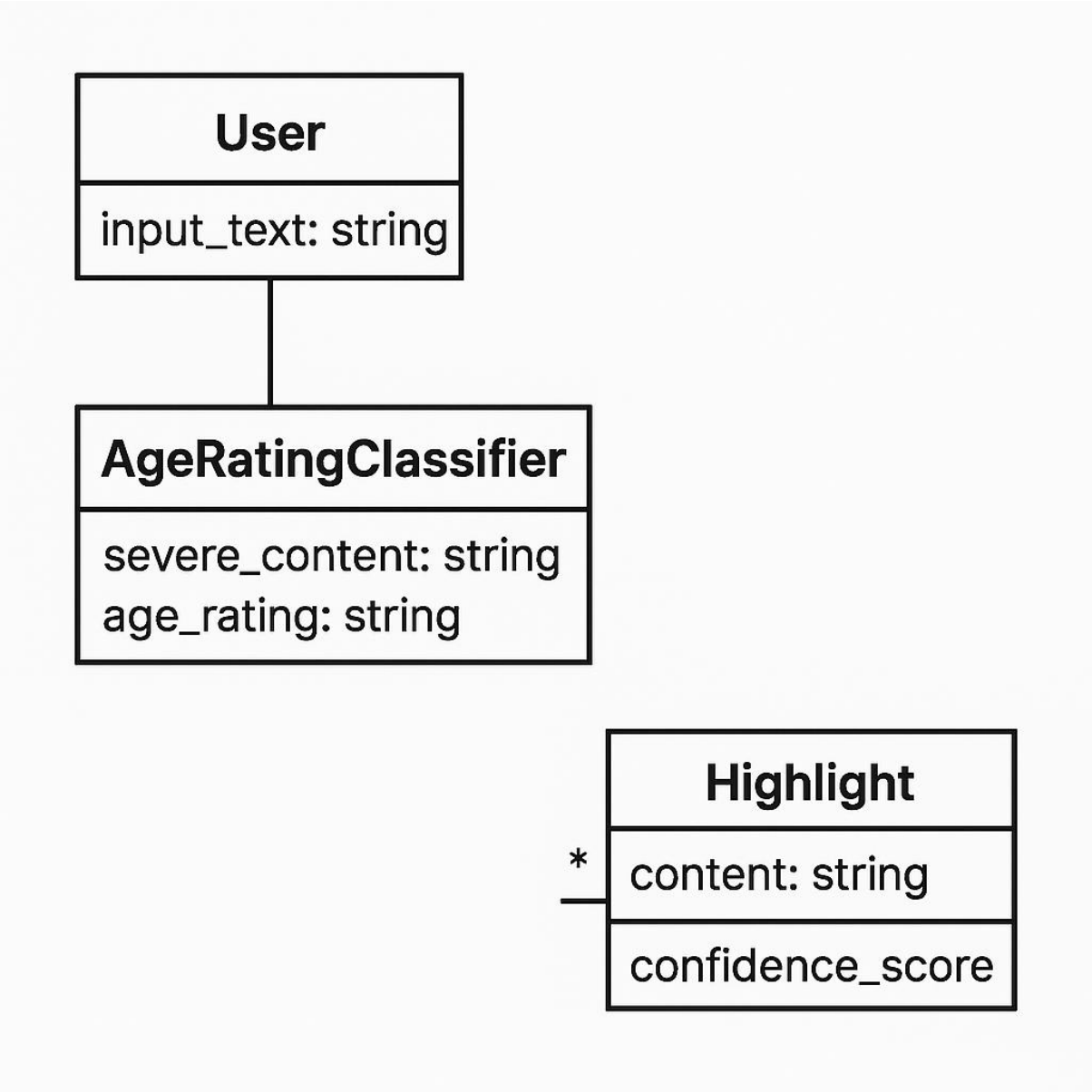

Figure 2

Table 2 - Non-Functional Requirements

ID	Requirement (User/System)	Priority	Type
11	The system must align age rating predictions with official UK BBFC guidelines.	Must	System
12	The system must maintain user privacy and not store any personal data.	Must	User

13	The system should be able to run on Google Colab or similar environments with GPU acceleration.	Should	System
14	The model's accuracy must be over 50% on test data.	Must	System
15	The interface must be accessible and easy to use with no prior training required.	Must	User
16	The output must be explainable and interpretable by end-users.	Must	User
17	The UI accommodates issues such as color blindness	Must	User
18	The application must perform preprocessing using SpaCy for sentence-level formatting.	Must	System
19	The solution must be compatible with modern web browsers (e.g., Chrome, Edge, Firefox).	Must	System
20	The system won't support collaborative annotation or crowdsourced feedback at this time.	Won't	User
21	The system must support efficient processing using GPU resources.	Must	System

Figure 3 – Environment diagram for Requirement #11

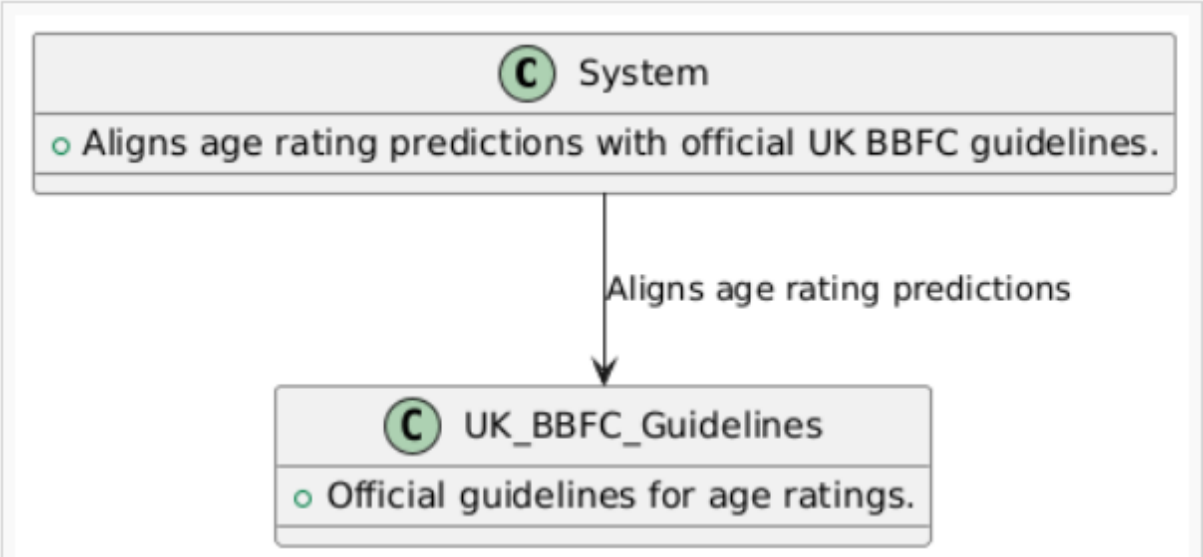

Figure 3

Figure 4 – Environment diagram for Requirement #21

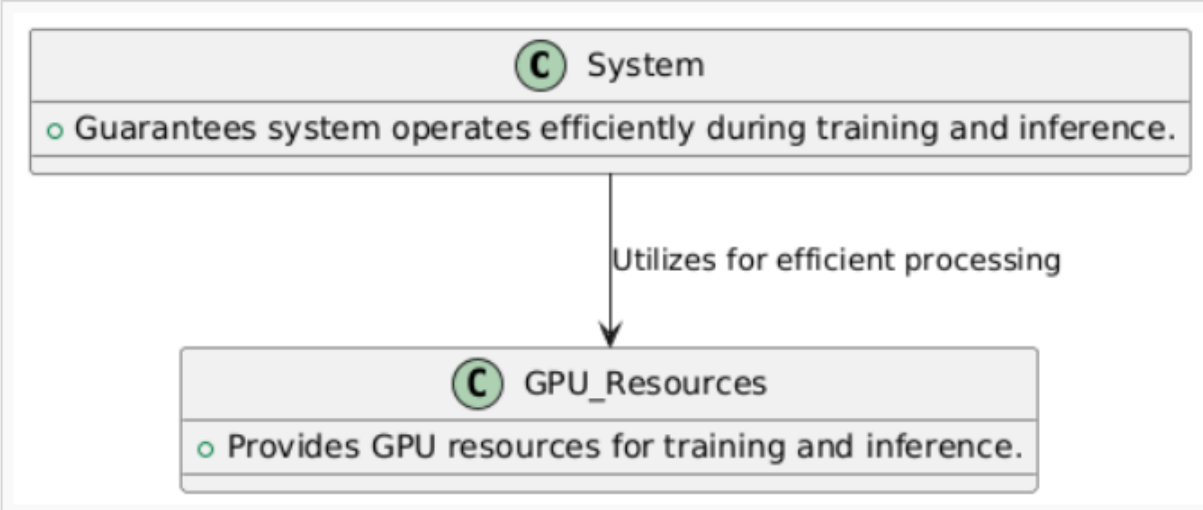

Figure 4

Methodology

This project followed an Agile Software Development methodology, which was chosen due to its flexibility and iterative nature. The main emphasis of Agile development is its frequent evaluation with continuous refinement, which aligned well with the training runs of this project. Since this project frequently applied new methods and techniques to improve model accuracy, sometimes even major changes such as switching the model that was being used, the Agile approach proved very useful in continuously testing and improving the model after each training run. It also allowed the project to adapt to new challenges that arose as throughout the development process. Iterations

were structured around short development cycles, each focused on improving specific aspects of the pipeline, including data processing, model architecture, and evaluation metrics.

The high-level objective of this project was to build a machine learning system that could accurately predict UK film age ratings from full movie scripts, using natural language processing (NLP). Early requirements included:

- Support for long text sequences.
- Classification into one of six UK age ratings (U, PG, 12, 12A, 15, 18).
- Proper handling of script formatting and metadata.
- Accurate evaluation using metrics like accuracy, F1 score, and AUC.

A product backlog was created with key tasks such as data cleaning, model selection, tokenization, chunking, training, and evaluation. Each iteration began with task planning, and progress was reviewed regularly to accommodate discoveries (e.g., token length issues with BERT requiring chunking and moving to Longformer).

Agile's iterative cycles were reflected in the step-by-step development of the system:

- **Data Preprocessing**: Movie scripts were scraped with unnecessary whitespace and empty files removed. The associated age rating was then added to the file, and any scripts without a valid rating were deleted. Custom chunking was developed to split long scripts into manageable 4096-token segments compatible with Longformer.
- **Model Selection and Adaptation**: The project initially employed BERT but shifted to Longformer-based models when BERT's token limit impacted performance negatively. Fine-tuning was applied using AutoModelForSequenceClassification with applied class weights and CrossEntropyLoss to account for label imbalance.
- **Training Pipeline**: The Hugging Face Transformers library was integrated with a custom Trainer class to include weighted loss functions and compute classification metrics. Hyperparameters such as batch size, learning rate, and weight decay were tuned iteratively.
- **Evaluation and Refinement**: After each cycle of training, models were evaluated using classification reports and AUC. Training results were saved to Excel spreadsheets and current issues were addressed in the next iteration.

Tools that were used for the project:

- **Programming Language**: Python
- **Libraries**: Hugging Face Transformers, PyTorch, Datasets, Scikit-learn, spaCy, pandas, Streamlit, numpy, BeautifulSoup, os, re, heapq, evaluate
- **Model Architectures**: BERT, Longformer
- **Evaluation Tools**: ROC-AUC, F1 Score, Precision, Recall, Accuracy, Classification Report
- **Development Environment**: Jupyter notebooks (in Google Colab), Python scripts, Excel (for results export)

Agile encourages continuous testing, which was implemented through evaluation after each training epoch using various metrics for each individual age rating. Validation sets were created via stratified

splits to ensure label balance. All experiments were logged and saved in structured formats for analysis and comparison across iterations.

Generally, using Agile allowed the project to remain adaptive to several challenges (e.g., long sequence handling, class imbalance) and allow for structured solutions and tests. It supported rapid prototyping, regular evaluation, and incremental learning, which was integral to the success of this project.

Design

For the design of this project, the high-level design choices were considered first. An aspect of this high-level design is system architecture, of which there are 4 main components to consider. First is the collection of the dataset for this project. To get a substantial set of complete movie scripts, the springfieldspringfield website was used, which contains over 44,000 movie scripts. To collect these into a usable dataset, scraping tools were used to save each movie script to its own file. Naturally, downloading and using 44,000 different scripts would be both too time consuming and computationally expensive for the scope of this project, so a compromise was made with how many scripts would be used. Further scraping was then performed on the BBFC website to assign a UK age rating to each script via appending it to the first line of each file.

The second aspect of the system architecture to consider is the preprocessing of the script text into a usable form for the model training to use. This involves the tokenization of the movie script, as well as using a pre-trained SpaCy NLP to transform the segmented lines of the movie scripts into more structured sentences to help the model's contextual understanding of text. The third system architecture component is the training and evaluation of the model itself. The system uses a Longformer model, and this component involves taking the pre-processed movie scripts and splitting them into a training, testing, and validation set. This component is the brunt of the project and is where most of the time and focus was spent. Once an initial model was set up and ready to be fine-tuned, several different approaches were used to improve the accuracy of the model, as well as a lot of hyperparameter tuning patterns. The validation data was used to provide quick accuracy results while training, and the model was run on the test data at regular intervals to check for problems like overfitting.

Finally, the last aspect of the system architecture is the UI using Streamlit. Streamlit was chosen due to its ease of use, with it being described as "simple and intuitive, making it ideal for data scientists and machine learning engineers to build and share their work" (Streamlit.io, n.d.) A simple web page was created that can have a file uploaded to it and then displays a predicted age rating along with some more metrics. This is also where the model will identify the lines in the script with the most influence on the age rating and print them out highlighted for the user to see. Figure 5 shows the system architecture diagram for the project, where the discussed aspects of the project can be seen including the data preprocessing and collection, the UI, and the model training. Figure 6 shows a wireframe for how the UI will look, with clear sections for file uploading and output for age rating predictions and highlighted text.

Figure 5 – System architecture diagram

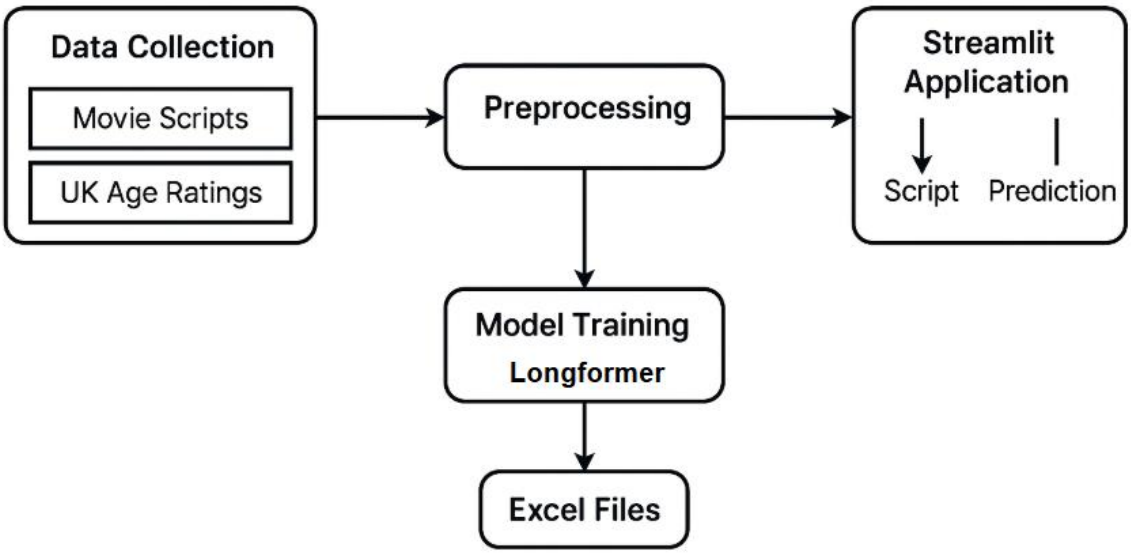

Figure 5

Figure 6 – UI Wireframe

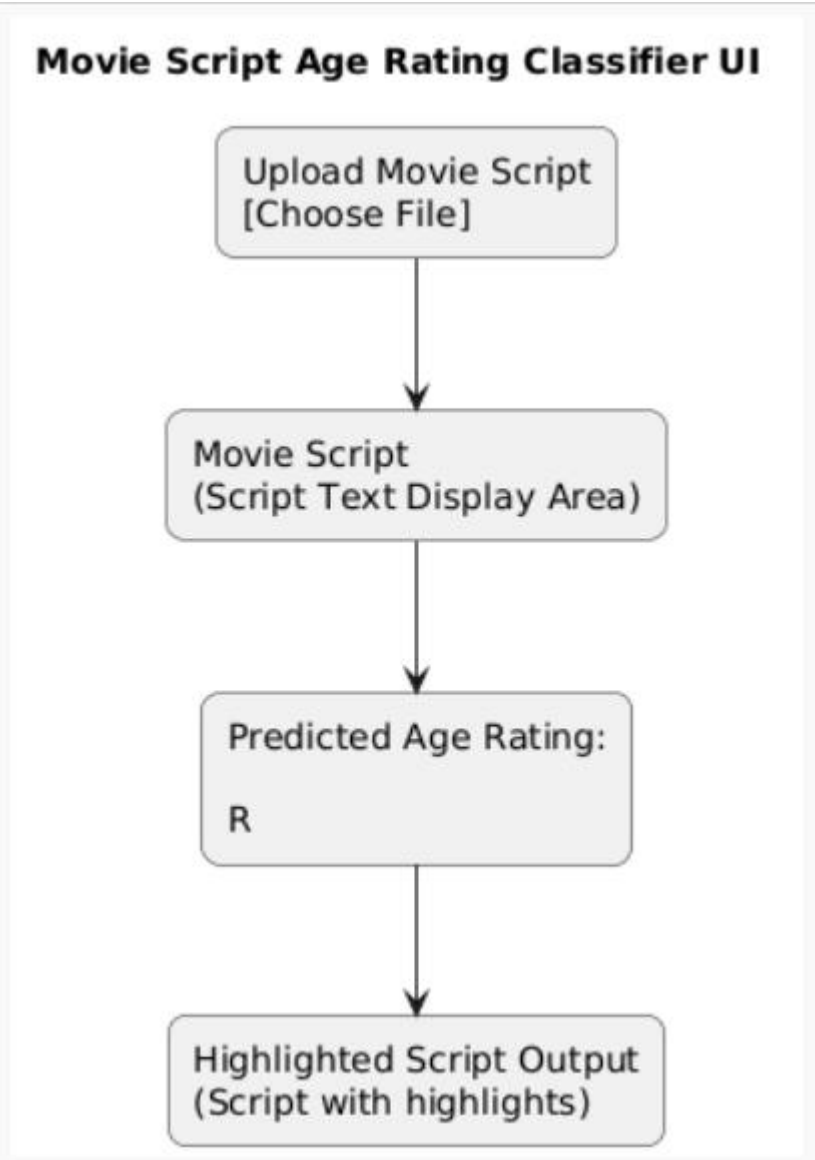

Figure 6

As far as data storing is concerned with this project, there was no need for any advanced method or database usage. The dataset was stored locally in a designated folder as .txt files, and the trained model was saved using checkpointing. Furthermore, the environment design for this project includes several external sources that were integrated with the project design. The obvious being the dataset sources (BBFC and springfieldspringfield websites) which were scraped to create the dataset. Moreover, both the pre-trained SpaCy model and from the spacy library and the pre-trained longformer model from the HuggingFace library were used. While SpaCy was used directly to run on the script, the pre-trained longformer model (the longformer-base-4096 model from allenai) needed to be further trained and fine-tuned for use on the dataset, which even meant overwriting pre-made classes and functions. Training and testing metrics such as accuracy was also saved and exported as Excel spreadsheets after running was complete. Next, as mentioned, the model UI was built on Streamlit. The Streamlit library will be used to create a basic looking webpage, and the free cloud options will be used to host the site. Finally, the paid version of Google Colab was used to train the data, as the computational requirements were simply too high to run on a local machine with no GPU, whereas Google Colab usefully gives access to an A100 GPU with high RAM on the Pro version.

Figure 7 – environment diagram for the project

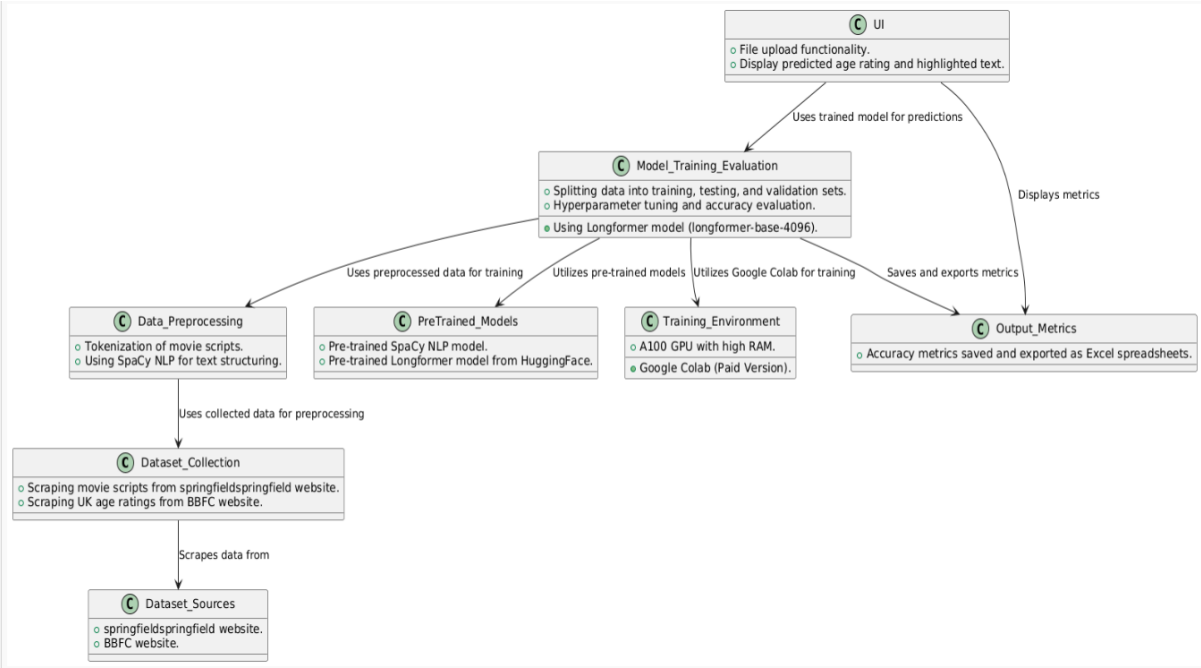

Figure 7

The project will make no use of OOP principles as they are not necessary for the task it is carrying out. Instead, the best model of a training run will be saved as a checkpoint which can be loaded in as a file when the model needs to be used. The project is mostly built relying on functions that are used to process text into the correct form and scrape data. Since each file is relatively self-contained and the finished model can be saved as a file, there is no need for anything to be represented as objects.

For the low-level design of the project, the layout of the UI has been considered deeply. It will be split into three main sections. One of these sections is a file upload box for users to upload a txt file for the model to be run on it, as well as providing a brief excerpt of the file in a text box, so users can see what they have uploaded for quality of life in case the user accidentally uploads the wrong file. Next, a predicted age rating will be displayed in bold for clarity, and a bar chart will be generated based on the confidence scores for each age rating label from the model. This bar chart will allow users to hover over each bar and see the exact value, as well as zooming in and saving the image. This and the bold text are accessibility features for people with poor eyesight. Colour blindness will also be accommodated for, as only contrasting darker/lighter colours will be used on the site, in line with Requirement #17. Finally, the movie script will be printed out in full, with the top 30 most influential lines highlighted in orange.

For the implementation of this project, several pre-developed tools will be used, such as HuggingFace Transformers, SpaCy NLPs, the longformer-base-4096 model, an Streamlit. HuggingFace provided many functions for use with this project as it is a “community-driven hub that offers tools and pre-trained models for natural language processing (NLP)” (HuggingFace.co, n.d) which makes it very specialised for this kind of task. Each of these tools will be used to enhance the running and training of the model and will be fine-tuned to the specific needs of the project accordingly. On top of this, there are also several new elements to the code that will be developed entirely from scratch, such as the scraping tools to gather the dataset, the highlighting logic for finding the most influential lines, and the fine-tuning methods that will be taken to enhance the accuracy of the model.

Finally, there will be several different testing stages for the project. The main testing stage will be in the form of validation data that is tested during training. This will give a good idea of model metrics such as accuracy and AUC, and reports will be printed out to the terminal after each epoch to show these metrics for each rating so that phonemena like over or underfitting can be identified and a general idea of which ratings the models succeeds and fails at can be established. Once training is complete, the model will then be run on test data to give another accuracy metric, and the number of correct and incorrect predictions for each rating will be displayed, so that patterns can be identified and worked on. Finally, integration testing will verify that the Streamlit app works correctly with the model by visually checking output in both the terminal and Streamlit page for both the predicted rating and highlighted lines. Then, individual unseen movie scripts will be used to gauge the finished model's performance on Streamlit. Overall, the testing will mainly focus on prediction accuracy, with some room for UI responsiveness.

Implementation

The implementation of this project first began with setting up the scraping tools via Python's BeautifulSoup module to gather a dataset. While the springfieldpspringfield site was used, the dataset was limited by time and computational limitations. To get the 40,000+ scripts available on the site would require a massive amount of time, as well as increasing the computational power needed to train the model by a wide margin, which was unsuitable for the scope of this project. Instead, a random page number was selected and the scraping tool combed through the next 100 pages gathering all the scripts from them. The structure of the website is such that each page contains a list of links to movie scripts, where the links take the user to a page with a text box that has the script inside, as seen in Figures 8 and 9. The scraping tool used made use of a loop to go through each title (which includes the year of release) and created txt files with the same title. While doing this, it then followed the link for each title and copied the contents of the text box into its respective file. Though this took a while to run, it created a large dataset for the project to use.

Figure 8 – search page on SpringfieldSpringfield

Figure 8

Figure 9 – script page on SpringfieldSpringfield

Figure 9

After the gathering of the data was done, it was then important to assign the UK age rating for each movie to the script. This was also achieved via scraping tools, this time using the BBFC website to find each movie. The site has a search functionality to find movies, which meant that the scraping tool could loop through the folder with each script in and search for its title. As shown in figure 10, the search results for a given movie include its UK age rating on the page, which could then be scraped. The year from the title was first checked to see if it matched the one shown on BBFC, then the rating was found and appended to the top line of each file, simply returning “no matching title/year found” if the search didn’t return anything. However, this did finish with many unsuitable scripts, for which the rating wasn’t found. To fix this, the data was cleaned by combing through each file and searching for a line beginning with “UK Age Rating:”. If it was found, the rating was compared to a list of valid ratings (U, PG, 12, 12A, 15, and 18) and if either the line wasn’t found or the rating was invalid, the file was deleted. An issue with some empty files remaining due to the nature of looping through each line to find the rating line arose and was dealt with by adding a mock line to the top of each file, meaning that empty files would be caught as having no valid rating.

Figure 10 – BBFC search page

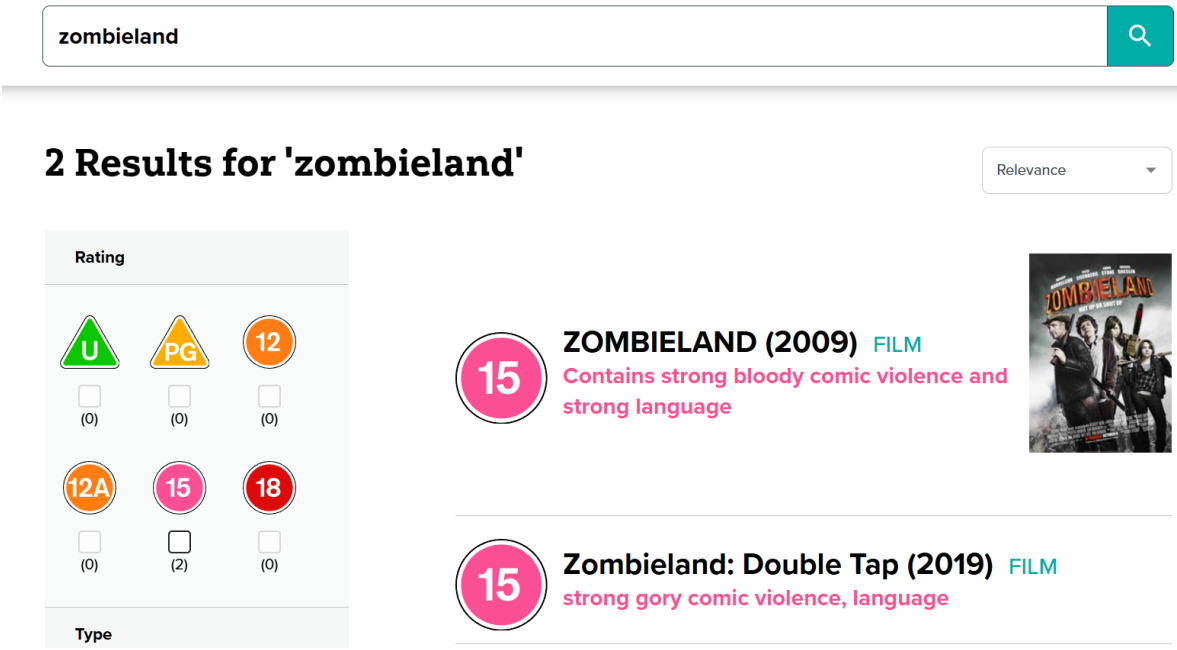

Figure 10

After the gathering and cleaning of the dataset was done, it was then time to train the model for this project. To begin with, the BERT-base-uncased model from google-BERT was used. A map of each valid rating to a numbered value (0-5) was made, and then the rating line of each script was found and used to make a list of dictionaries mapping the script text to its rating. The rating line itself was also removed in this dictionary, so that the model wouldn't be able to see it and use it to predict a rating. A pretrained tokenizer was set using the Transformers library so that it could tokenize parts of the script for model training. The data was then split into a training, testing, and validation set. First, the data was split to have 80% of it be used for training, and 20% for testing. The training set was then further split into 90% training and 10% validation, where the data was stratified to give representation to ratings with a smaller dataset. The Dataset library was used to take the scripts in the datasets and set their input_ids (the text of the scripts) and their labels (the age ratings). A pre-trained model was then used and given the age rating labels. BERT pre-trained models are already trained on identifying context within text incredibly well, meaning that to teach it the difference in age ratings, only fine-tuning was required. Due to computational limitations, most of the model layers were frozen, with only the pooler layer and classification head being unfrozen. BERT models use a massive number of layers, meaning that keeping all of them active, while better for leaning, had massive computational requirements.

The Numpy and Torch libraries were then used in a compute_metrics function to calculate the predictions using softmax and map them to their classes using argmax. Metrics such as accuracy and AUC were taken from the Evaluate library and used to display stats about the model's performance after each epoch. Finally, a trainer was set and provided with the data, classes, tokenizer, and compute_metrics function. After training, the log history of the trainer after each epoch was then appended to an Excel spreadsheet to keep track of the results of each training run. On top of this, the scores for each rating were also displayed after each epoch. Figure 11 shows the results of the first training run. As shown, the accuracy and AUC scores were very bad, with the AUC only reaching 50% (which is no better than random guessing). Upon checking the individual rating scores, it was found that the model had overfitted to the 15 rating. The dataset was checked, and it was found

that there were substantially more scripts rated 15 than any other rating, which had taught the model that guessing 15 every time gave it a 50/50 chance to get the rating right, meaning it was never guessing anything different. Figure 12 shows one of these outputs where 15 was the only rating being guessed.

Figure 11 – initial training results

epoch	accuracy	precision	recall	f1	auc
1	0.357143	0.127551	0.357143	0.18797	0.492
2	0.357143	0.127551	0.357143	0.18797	0.53
3	0.357143	0.127551	0.357143	0.18797	0.549
4	0.357143	0.127551	0.357143	0.18797	0.549
5	0.357143	0.127551	0.357143	0.18797	0.557
6	0.357143	0.127551	0.357143	0.18797	0.551

Figure 11

Figure 12 – evidence of overfitting to 15

```
True class counts: Counter({np.int64(4): 75, np.int64(2): 69, np.int64(1):
              precision    recall  f1-score   support

           U       0.71      0.62      0.67        56
          PG       0.57      0.54      0.55        69
          12       0.57      0.62      0.60        69
         12A       0.68      0.66      0.67        29
          15       0.60      0.49      0.54        75
          18       0.57      0.80      0.67        46

    accuracy                           0.60       344
   macro avg       0.62      0.62      0.61       344
weighted avg       0.61      0.60      0.60       344
```

Figure 12

Upon identifying this issue, the training code was changed to make use of class weights to give harsher punishments for guessing more frequently seen ratings and higher rewards for less frequent ones. Class weights are very useful when dealing with data imbalance, as they help “mitigate the bias introduced by the majority class and allows for more effective learning of minority class characteristics” (Chawla, Kegelmeyer & Bowyer, 2002). To achieve this, the number of samples from each rating was counted and then the total number of samples was divided by the number from each rating. This meant that the inverse frequency of each rating was counted to calcuate the class weights. The Trainer function was then overwritten by a new class – WeightedTrainer. In this new class, the compute_loss method was overwritten to use CrossEntropyLoss (from torch.nn) with the provided weights. Figure 13 shows the code of this overwritten function. The results after training can be seen in Figure 14, where the overfitting issue has been resolved. However, the AUC score has remained around the same and the accuracy has dropped massively. The highest accuracy score was 19%, which, with 6 labels, is no better than random guessing. This showed that the model was not

learning the intricacies of the scripts and was only had guessing 15 to rely on for getting a decent accuracy.

Figure 13 – new WeightedTrainer class and overwritten compute_loss function

```
# overwriting the Trainer class
class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.get("labels")
        outputs = model(**inputs)
        logits = outputs.get("logits")
        # set the loss function to include weights
        loss_fct = CrossEntropyLoss(weight=class_weights.to(model.device))
        loss = loss_fct(logits, labels)
        return (loss, outputs) if return_outputs else loss
```

Figure 13

Figure 14 – Training results after adding class weights

epoch	accuracy	precision	recall	f1	auc
1	0.190476	0.054916	0.190476	0.084249	0.51
2	0.142857	0.177262	0.142857	0.136464	0.592
3	0.214286	0.168878	0.214286	0.187785	0.625
4	0.142857	0.073982	0.142857	0.093339	0.633
5	0.190476	0.147781	0.190476	0.153756	0.635
6	0.190476	0.17797	0.190476	0.166545	0.635

Figure 14

After looking at the dataset again, the inconsistency in the representation of each rating was proving to be a big problem. The dataset had a severe lack of 12A scripts, and an overwhelming excess in 15 ones, which was leading to problematic training, as the model simply didn't have enough data to work with. After identifying this, the dataset was forcefully balanced by choosing a maximum number of samples (250) for each rating and deleting any scripts that stepped over this maximum. While the excess in 15s and underrepresentation of 12As still existed, the gap was now much smaller than it previously had been, and the other ratings were almost all equally represented. Figure 15 shows the balanced number of scripts for each rating. The training results after balancing the dataset are shown in Figure 16, where the accuracy had improved to be between 35-40%. While still not great, the model had now reached the level it was at while overfitting but this time by predicting correct age ratings. While this was a step in the right direction, there were still changes to be made to improve the accuracy of the model.

Figure 15 – the number of scripts present for each rating

```
PS C:\Users\jtove\Documents\GitHub\DSP\DSP> python -u "c:\Users\jtove\Documents\GitHub\DSP\DSP\balancingDataset.py"
Total files for each rating: {'U': 138, 'PG': 193, '12': 234, '12A': 93, '15': 225, '18': 200}
```

Figure 15

Figure 16 – Training results after balancing the dataset

epoch	accuracy	precision	recall	f1	auc
1	0.25	0.166667	0.25	0.188312	0.622
2	0.409091	0.296023	0.409091	0.332442	0.693
3	0.363636	0.501873	0.363636	0.348359	0.723
4	0.363636	0.372376	0.363636	0.359472	0.738
5	0.363636	0.380441	0.363636	0.361493	0.739
6	0.340909	0.348623	0.340909	0.333317	0.742

Figure 16

To improve the accuracy of the project, a couple of changes were made. The first being the switch in model path from google-BERT's BERT-base-uncased to allenai's longformer-base-4096. Part of the reason the model was suffering from inaccuracies was due to BERT's maximum token limit – 512. This meant that only the first small section of each script was being fed to the model, meaning it was likely seeing many scripts with relatively inoffensive beginnings, that only later on fell more in line with their age ratings. The advantage of switching to Longformer was that it had a maximum token limit of 4096 as it uses a "sparse attention mechanism based on the local window attention pattern, allowing it to scale to documents of up to 4,096 tokens" (Beltagy, Peters & Cohan, 2020), which is enough to cover almost half of most scripts. However, the computational requirements to run a model like this on the multiple thousands of scripts in the dataset was far too large and impractical to run on a standard laptop's CPU. It was for this reason that the implementation of this project shifted from using vscode running on a CPU, to using the paid version of Google Colab, making use of their high RAM and A100 GPU, which allowed the model to run far more efficiently and within a reasonable time frame.

The second of these changes was making use of SpaCy for text preprocessing. As shown by Mohamed and Ha in their 2020 paper 'A first dataset for film age appropriateness investigation', the springfieldspringfield dataset was written to macth screens, and thus was "not made up of valid linguistic entities". The pre-trained SpaCy NLP was used to tokenize the script into understandable sentences, so that longformer's contextual understanding could be used more effectively. Figure 17 shows the code for using SpaCy to preprocess script text. In it, a SpaCy pipeline was used to batch process scripts for efficiency, and RegEx was used to format newlines correctly. The texts were then stripped of any whitespace and returned along with their age rating. After training the model with these new changes, general accuracy increased to be consistently at 39-40%, however the best performance had not been topped. It seemed that the model had improved only very slightly by these changes, which was quite surprising given the increase in token length and script preprocessing. After looking into what could be preventing the model from getting any more accurate, it became clear that freezing most of the model's layers was having a large effect on the accuracy. A large amount of learning could be performed in these layers, and only relying on the pooler layer and classification head was severely limiting the model. Figure 18 shows the training results after using Longformer with SpaCy.

Figure 17 – Code for SpaCy pipe processing

```python
def process_all_scripts(scripts):
    # clean scripts and prepare text-label pairs
    cleaned_scripts = [clean_script(script) for script in scripts]

    # extract raw texts and labels
    raw_texts = [s["raw"] for s in cleaned_scripts]
    age_ratings = [s["label"] for s in cleaned_scripts]
    cleaned_texts = []
    for script in raw_texts:
      # clean the scripts into the desired format
      subbed_text = re.sub(r"(?<!\n)\n(?!\n)", ' ', script)
      cleaned_texts.append(subbed_text)

    # run the pipeline on the cleaned scripts
    docs = list(nlp.pipe(cleaned_texts, batch_size=64))
    processed_scripts = []
    for doc, label in zip(docs, age_ratings):
        # combine sentences back into one bit of text
        sentences = [sent.text.strip() for sent in doc.sents if sent.text.strip()]
        cleaned_script = "\n".join(sentences)
        processed_scripts.append({"text": cleaned_script, "label": label})
    return processed_scripts
```

Figure 17

Figure 18 – Training results after switching to Longformer and using SpaCy

epoch	accuracy	precision	recall	f1	auc
1	0.272727	0.089765	0.272727	0.135019	0.708
2	0.340909	0.335184	0.340909	0.307229	0.767
3	0.409091	0.426362	0.409091	0.350539	0.797
4	0.397727	0.372091	0.397727	0.342349	0.812
5	0.420455	0.407927	0.420455	0.364591	0.815

Figure 18

After unfreezing all model layers, layer-wise learning rate decay was also implemented. To do this, an initial lowest learning rate was set for the embedding layer and were then gradually increased across the encoder layers before giving the highest learning rate to the classifier layer. Figure 19 shows the code for implementing the layer-wise learning rate decay. The reason it was used was to prevent catastrophic unlearning, which was a large concern as “catastrophic forgetting (or unlearning) occurs when a model trained on one task forgets previously learned tasks due to the interference from new learning” (Zenke, Poole, & Ganguli, 2017). Altering the learning rate also served as hyperparameter tuning to find the best initial settings for the model. Now that each layer was being used, the decay of the learning rate ensured that each layer was both improving on the last while not forgetting their pre-training. Figure 20 shows the training results after training with all the layers unfrozen, where the model’s accuracy managed to reach 50%, a full 10% increase from the last training run, which is in line with Requirement #14.

Figure 19 – Code for layer-wise learning rate decay

```python
# Add embedding layer with lowest LR
optimizer_grouped_parameters.append({
    "params": [p for n, p in model.longformer.embeddings.named_parameters() if not any(nd in n for nd in no_decay)],
    "weight_decay": weight_decay,
    "lr": initial_lr * (layerwise_lr_decay ** num_layers)})
optimizer_grouped_parameters.append({
    "params": [p for n, p in model.longformer.embeddings.named_parameters() if any(nd in n for nd in no_decay)],
    "weight_decay": 0.0,
    "lr": initial_lr * (layerwise_lr_decay ** num_layers)})

# Add encoder layers with decaying LRs
for i, layer in enumerate(encoder_layers):
    layer_lr = initial_lr * (layerwise_lr_decay ** (num_layers - i - 1))
    optimizer_grouped_parameters.append({
        "params": [p for n, p in layer.named_parameters() if not any(nd in n for nd in no_decay)],
        "weight_decay": weight_decay,
        "lr": layer_lr,
    })
    optimizer_grouped_parameters.append({
        "params": [p for n, p in layer.named_parameters() if any(nd in n for nd in no_decay)],
        "weight_decay": 0.0,
        "lr": layer_lr,
    })

# Add the classifier layer
optimizer_grouped_parameters.append({
    "params": [p for n, p in model.classifier.named_parameters() if not any(nd in n for nd in no_decay)],
    "weight_decay": weight_decay,
    "lr": initial_lr})

optimizer_grouped_parameters.append({
    "params": [p for n, p in model.classifier.named_parameters() if any(nd in n for nd in no_decay)],
    "weight_decay": 0.0,
    "lr": initial_lr})
```

Figure 19

Figure 20 – Training results after unfreezing layers and adding layer-wise learning rate decay

epoch	accuracy	precision	recall	f1	auc
1	0.386364	0.27566	0.386364	0.307341	0.811
2	0.454545	0.387158	0.454545	0.35201	0.841
3	0.545455	0.432754	0.545455	0.465467	0.853
4	0.477273	0.373309	0.477273	0.409413	0.853
5	0.5	0.4835	0.5	0.470267	0.855

Figure 20

Finally, the last adjustment made to this project was splitting each movie script in the dataset into chunks. Despite all the progress made, it seemed as though the dataset was still lacking in size, and the token limit on Longformer, while larger than BERT, was still having issues with getting the full context of the script. While there were no computationally feasible models to use to increase the token limit, splitting the data into chunks was the next best solution. Figure 21 shows the code splitting each script into chunks with a length of 4096 (adding padding if they didn't reach that limit) and then assigning a script's age rating as a label to each chunk of that script. This artificially bolstered the dataset with a significant increase in data and being that it all came from the original set of scripts, the dataset was still balanced. Figure 22 shows the final training run, in which the accuracy once again jumps up by 10%, this time reaching 60%. At this point, it seemed as though there was nowhere for the model to go, and this was for several reasons. One such reason being that the dataset was still simply too small compared to the datasets used on most other NLP tasks,

and increasing it even further was simply impractical, as even on an A100 GPU, OOM (Out Of Memory) errors still cropped up unless cache was emptied during training and gradient checkpointing was enabled, aligning with Requirement #21. Even then, it took over an hour to run, meaning that a more powerful model or significantly larger dataset was too impractical. Furthermore, the nature of the dataset also played a factor in the accuracy. The dataset contained a wide range of movie scripts from various years, from the 50s up to 2025, meaning that the model was potentially being given scripts with outdated age rating rules, as standards for ratings has changed drastically over the years.

Figure 21 – Code for chunking the scripts

```python
# split the script into multiple chunks
def chunk_script(text, tokenizer, max_length=4096, stride=512):
    tokens = tokenizer(text, return_attention_mask=False, return_token_type_ids=False, truncation=False)["input_ids"]
    chunks = []
    for i in range(0, len(tokens), max_length - stride):
        chunk = tokens[i:i+max_length]
        if len(chunk) < max_length:
            # pad the chunk if 4096 tokens isn't met
            chunk += [tokenizer.pad_token_id] * (max_length - len(chunk))
        chunks.append(chunk)
    return chunks

# create a dataset from the dictionary of chunked scripts
def create_chunked_dataset(processed_scripts, tokenizer):
    chunked_texts = []
    chunked_labels = []

    for script in processed_scripts:
        chunks = chunk_script(script["text"], tokenizer)
        chunked_texts.extend(chunks)
        chunked_labels.extend([script["label"]] * len(chunks))

    return Dataset.from_dict({"input_ids": chunked_texts, "label": chunked_labels})
```

Figure 21

Figure 22 – Training results after chunking scripts

epoch	accuracy	precision	recall	f1	auc
1	0.465116	0.404506	0.465116	0.430356	0.828
2	0.517442	0.52165	0.517442	0.512782	0.861
3	0.555233	0.557778	0.555233	0.547085	0.876
4	0.59593	0.601395	0.59593	0.595681	0.886
5	0.604651	0.60889	0.604651	0.602201	0.887

Figure 22

After the model was trained sufficiently, it was then run on test data acquired from the train test split at the beginning of the code. The model achieved a similar accuracy to the training data of 59% when run on test data. Figure 23 shows the first few lines of the results of the test saved to an Excel spreadsheet (0=U, 1=PG, 2=12, 3=12A, 4=15, 5=18). Initially, it looked like the model was mainly mixing up movies with a rating of 12A, as it was classifying them as either 12 or 15. However, after finding the number of incorrect predictions for each model and comparing them to the number of correct ones, it appeared that 15 was being guessed incorrectly the most. This could be due to the changes in rating standards over the years as it appeared that a lot of 15s were being guessed as 18s and also to do with its imbalance in the dataset leading to its class weight punishing mistakes on

guessing 15 harsher. On the other hand, the model seemed adept at guessing Us and PGs, getting much more right than wrong, likely due to their distinct nature in not containing much offensive content. Figure 24 shows the number of correct/incorrect predictions for each rating:

Figure 23 – Test predictions

true_label	predicted_label
0	0
4	5
2	3
1	1
3	3
5	5
5	5
2	2
4	5
1	1
3	3
1	1
5	3
5	2
3	3
5	5
0	0
4	2
5	5
5	4
4	4
0	0
2	4
1	0
4	4
4	1
1	1
1	1
1	1
0	1

Figure 23

Figure 24 – number of correct/incorrect predictions for each rating

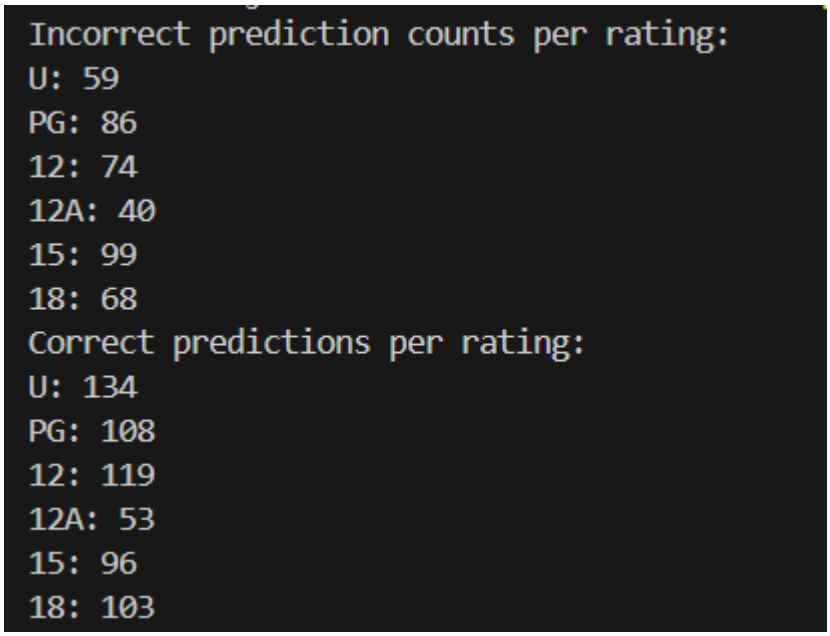

Figure 24

With the model trained and tested, the final step was to create a simple Streamlit UI to run the model. Due to Streamlit's user friendly nature, little input was needed to create a basic screen with the option to upload a txt file so that the model could be run on it. To start with, the model was simply called on a provided script (only taking the first 4096 tokens) to predict its age rating, however, it became apparent that this wasn't as accurate as the model could be. When provided with a 15 rated script (one that included a lot of mature content in particular) it guessed the script to be an 18, the same mistake that was made frequently on the test data. However, after splitting the whole movie script into 4096 token chunks and then finding the mean for each rating's confidence score to use, it correctly identified the script as being an 18 rated movie. This led to the finished model being seemingly more accurate with provided scripts. A pandas dataframe was used to build a bar chart for each age ratings confidence score to be displayed on screen. Figure 25 shows the Streamlit interface after predicting an age rating on a provided script. It can be seen in this figure that the UI is easily readable with clear input/output boxes (Requirements #15 and #16) and a clear graph showing the confidence scores is displayed, which is downloadable (Requirements #3 and #9)

Figure 25 – Streamlit UI giving a recommended age rating

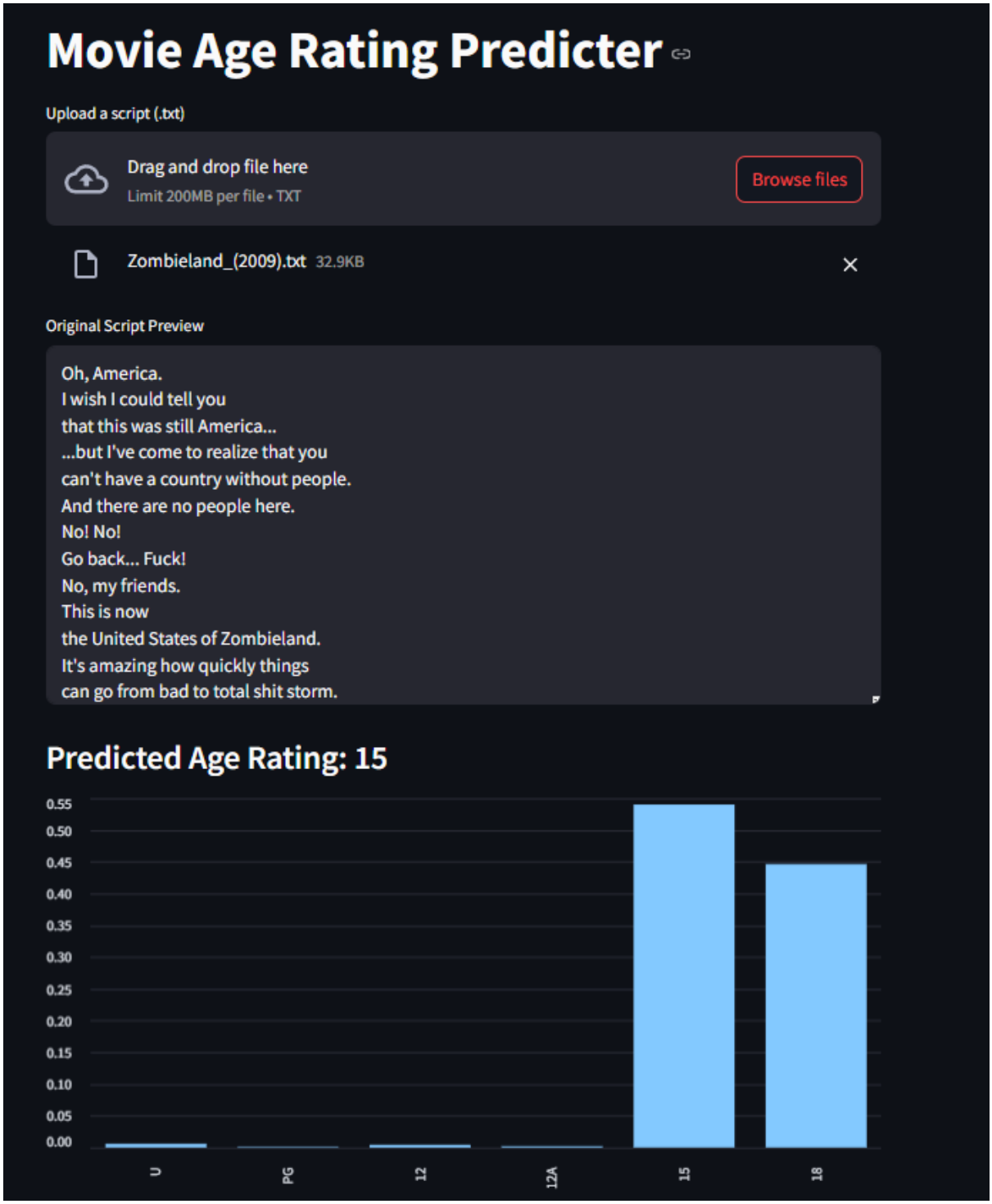

Figure 25

Then, the model needed to correctly identify the lines it felt impacted the final age rating the most. In order to make this possible, the model was run in batches on each line for the whole script to return an age rating and confidence scores for each. As a minimum threshold for a valid confidence score was tricky to define, the top 30 lines with the highest confidence scores the same as the script's predicted rating were found and highlighted. At first, it seemed as though the model was only detecting lines with profanity, as context was hard to gain only being given a line at a time, but when the model was run on batches of 5 lines instead of each at a time, it began identifying lines

carrying connotations of mature topics beyond just profanity, as shown in Figure 26, which is in line with Requirement #4.

Figure 26 – First few lines of highlighted influential lines on the UI

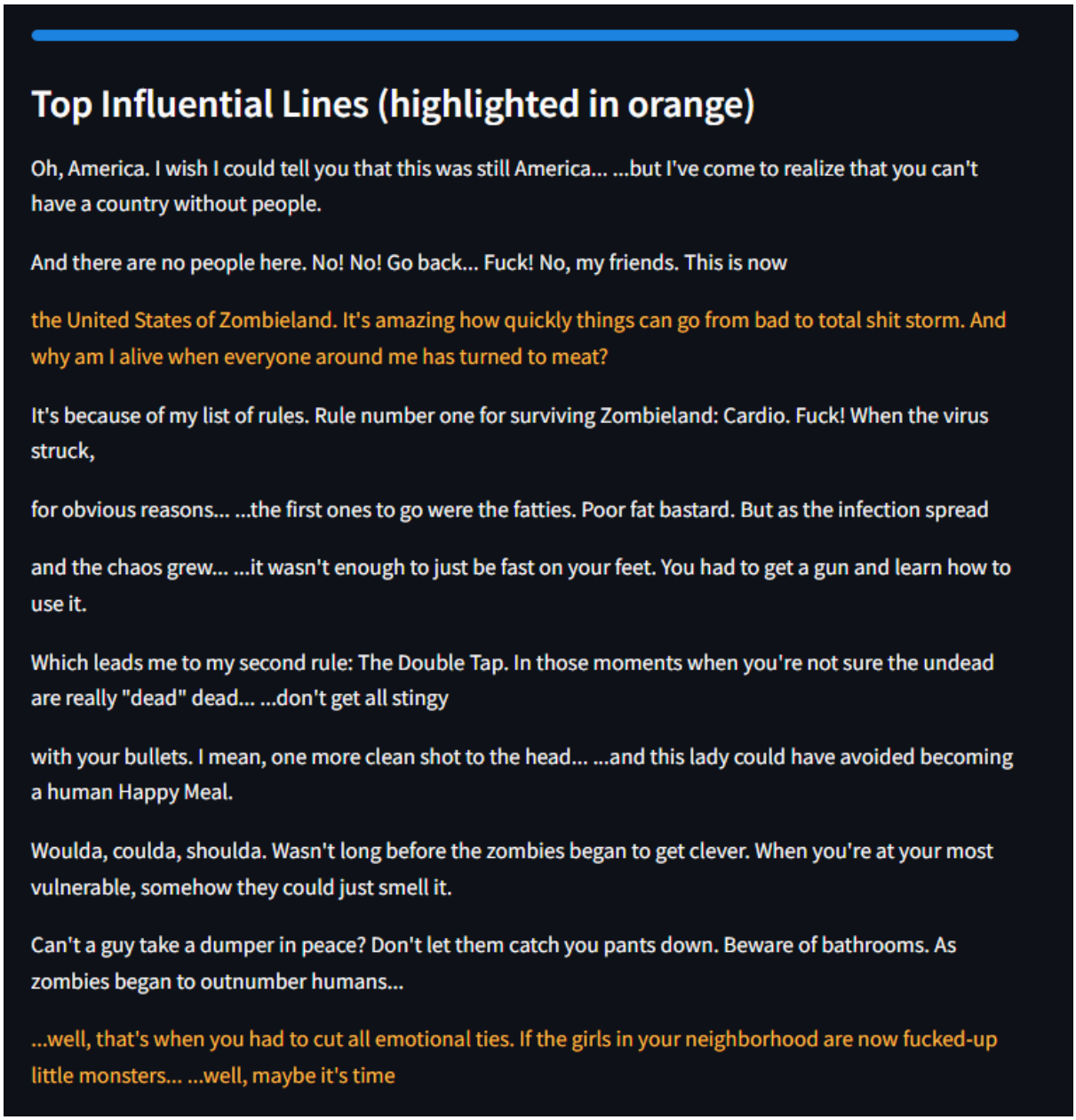

Figure 26

Project Evaluation

Overall, this project proved to be a very insightful study into complex machine learning tools and methodologies, as well as the practical applications for these tools. The requirements and design for the project were carefully considered after hours of broad research, eventually funnelling tools/methods of interest down to more specific concepts. Throughout the course of this research, a

large amount of information was learned and considered relating to constructing natural language processors, different machine learning models, and explicit content detection. All of this info was very insightful and helped to build a concrete list of requirements and set design. Even then, however, the project proved challenging with several setbacks and some aspects that changed from the initial background chapter (such as switching from BERT to Longformer).

Many limitations were encountered throughout the course of the project's implementation, particularly regarding time and computational power. The training results discussed throughout the implementation section demonstrate just how difficult it was to enhance the project's accuracy, with overfitting (Figure 12) being an initial issue due to the lack of balance in the dataset, and further issues being encountered with maximum size limits and text formatting. For later runs, the paid version of Google Colab was required to run the training, and even then it still took over an hour to complete the model's training due to the size and complexity of the dataset. While many fixes and enhancements were made, the accuracy was still ultimately capped at 60% (shown in Figure 22). There are several reasons for why this happened, with the most plausible being the inherent issues with the dataset. Despite collecting many samples and attempting to balance the dataset, imbalance was still a large issue, particularly with 12A rated scripts. Moreover, the subtle nuance between certain scripts of different age ratings led to a lot of room for error, especially with class weights sometimes influencing the model to guess 18 instead of 15 (implied by Figure 24). To improve the model, the quantity of data would need to be increased drastically (which was not possible for this project) meaning professional, industry standard machines with higher GPUs and processing power would be needed. All scripts in the dataset, if possible, should gathered from the last 10-15 years, as some scripts being from more than 3 decades ago had an impact on rating standards for this project.

Furthermore, the feedback received from meetings and the Project-in-Progress sessions were instrumental to the construction of this project and report. The feedback not only allowed for more a more professionally written report but also gave insightful comments for the research phase which gave a good head-start for the implementation of the model. It was also considered when developing a readable UI, which ended up playing an important role in the finished version of the project. While there is still plenty room for improvement on the project, particularly regarding the accuracy of the model, the finished product met all the requirements and proved a valuable learning experience with machine learning and NLP tools.

Further Work and Conclusions

In conclusion, this project was able to meet most of its initial aims. It developed a model that can predict UK age ratings upon being given a movie script with reasonable accuracy and highlighted the most influential lines of the scripts with a basic UI. Though the project struggled with accuracy at first, it was able to reach a good level of 60% by completion, which worked well for tested scripts. While the project did meet its requirements, there was still room for improvement with the final accuracy, as well as the scraping methods used to gather the dataset, which took a very long time. The outlined intended use of this project in the introduction was achieved by the final model, and the extra steps building on the background chapter considered in the design section regarding SpaCy and Streamlit were implemented to significant affect.

If future work was to be conducted on the project or if there was more time during the initial project window, it would have been good to be able to have enhanced the efficiency of the scraping tools, allowing for a larger and more balanced dataset, while also allowing for the deletion of any scripts falling outside of a certain year range. If access to higher power machines/tools could have been gotten, a much larger dataset could have been used to better affect with far less time constraints as well. Finally, an initial aim of the project was to identify reasons for the age rating (e.g. strong language), which was unfortunately scrapped by the requirements and design stages of the project when time constraints were considered. Tags corresponding to the reason for rating were available on the BBFC website, however, and with more time, this aim could have been met by using multi-label classification and training a second model to identify these tags from movie scripts and then using both developed models on the final UI.

Overall, the project did turn out very well and met most of its aims and requirements, and with more time could have expanded to an even higher level of accuracy with more overall features.

References / Bibliography

Gupta, H.G. and Patel, M.P. (2021) Method of Text Summarization Using Lsa and Sentence Based Topic Modelling with BERT. *International Conference on Artificial Intelligence and Smart Systems (Icais)* [online]., pp. 511-517. [Accessed 20 December 2024].

Zhao, H.Z., Phung, D.P., Huynh, V.H., Jin, Y.J., Du, L.D. and Buntine, W.B. (2021) *Topic Modelling Meets Deep Neural Networks: A Survey*. Australia: .

Juluru, K., Shih, H.-H., Keshava Murthy, K.N. and Elnajjar, P. (2021) Bag-of-words technique in natural language processing: A primer for radiologists. *Radiographics: a review publication of the Radiological Society of North America, Inc* [online]. 41 (5), pp. 1420–1426. Available from: http://dx.doi.org/10.1148/rg.2021210025.

Topper, N. (2023) *Bag-of-words model in NLP explained Built In*. 2 August 2023 [online]. Available from: https://builtin.com/machine-learning/bag-of-words [Accessed 24 January 2025].

Kasthuriarachchy, B.H., De Zoysa, K. and Premaratne, H.L. (2014) *Enhanced bag-of-words model for phrase-level sentiment analysis*. In: *2014 14th International Conference on Advances in ICT for Emerging Regions (ICTer)*2014/12/10-2014/12/13. IEEE, pp. 210–214.

Anon. (no date) *Datacamp.com* [online]. Available from: https://www.datacamp.com/blog/how-to-learn-nlp [Accessed 24 January 2025].

Kurama, V. (2020) *PyTorch vs. TensorFlow: Key differences to know for deep learning Built In*. 9 September 2020 [online]. Available from: https://builtin.com/data-science/pytorch-vs-tensorflow [Accessed 24 January 2025].

Novac, O.-C., Chirodea, M.C., Novac, C.M., Bizon, N., Oproescu, M., Stan, O.P. and Gordan, C.E. (2022) Analysis of the application efficiency of TensorFlow and PyTorch in convolutional neural network. *Sensors (Basel, Switzerland)* [online]. 22 (22), p. 8872. Available from: https://www.mdpi.com/1424-8220/22/22/8872 [Accessed 24 January 2025].

Mohamed, E. and Ha, L. (2020) A first dataset for film age appropriateness investigation. Calzolari, N. et al., eds. *International Conference on Language Resources and Evaluation* [online]. pp. 1311–1317. Available from: https://aclanthology.org/2020.lrec-1.164/.

Anton, C., Dmitry, I. and Preslav, N. (2021) *Transformers: 'the end of history' for NLP? arXiv [cs.CL]* [online]. Available from: http://arxiv.org/abs/2105.00813.

Sharma, S. and Joshi, N. (2024) A fusion approach to detect sarcasm using NLTK models BERT and XG Boost. *Journal of information & optimization sciences* [online]. 45 (4), pp. 981–990. Available from: https://tarupublication.s3.ap-south-1.amazonaws.com/articles/jios-1621.pdf.

Anon. (no date) *Streamlit.io* [online]. Available from: https://streamlit.io [Accessed 23 April 2025].

Anon. (no date) *Huggingface.co* [online]. Available from: https://huggingface.co [Accessed 24 April 2025].

Beltagy, I., Peters, M.E. and Cohan, A. (2020) *Longformer: The Long-Document Transformer arXiv [cs.CL]* [online]. Available from: http://arxiv.org/abs/2004.05150.

Cao, L., Mohan, K., Xu, P. and Ramesh, B. (2009) A framework for adapting agile development methodologies. *European journal of information systems: an official journal of the Operational Research Society* [online]. 18 (4), pp. 332–343. Available from: http://dx.doi.org/10.1057/ejis.2009.26.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) SMOTE: Synthetic minority over-sampling technique. *The journal of artificial intelligence research* [online]. 16, pp. 321–357. Available from: https://www.jair.org/index.php/jair/article/view/10302 [Accessed 24 April 2025].

De Lange Rahaf Aljundi Marc Masana Sarah Parisot, M. (2019) *Continual learning: A comparative study on how to defy forgetting in classification tasks Researchgate.net*. 2019 [online]. Available from: https://www.researchgate.net/publication/335908453_Continual_learning_A_comparative_study_on_how_to_defy_forgetting_in_classification_tasks [Accessed 30 April 2025].